

Lab Reflection — WalkMates

Lab: 1

Pair: Lena Edlind, Jakob Cederblad & Gemini CLI

Repo commit/tag: <https://github.com/leed2400/walkmates-test> / d36ace1a9a9614d6957ff00f5d252485053ae189

1. What we did

We did a quality-attribute analysis of three WalkMate features (FR-1.1, FR-3.1, and FR5-1). We identified two ISO/IEC 25010 quality characteristics that were most at stake and then defined a testable quality requirement for one of them.

We also proceeded with specification-based test design, using equivalence partitioning for FR-1.1 and FR-1.3, boundary value analysis for FR-1.3, and a decision table for the trust tiers in FR-1.2. We then implemented unit tests based on these test designs.

2. What we found

We found an error in the code. During test execution, our test for Swedish international phone numbers revealed that the code enforced the wrong length. This also showed us that AI may focus on modifying a test so that it passes rather than identifying the underlying bug in the code.

3. AI use (be honest — it doesn't lower your grade)

In Part A, we did not use AI. In Part B, we first created the specification-based test designs manually and independently of each other. We then asked an AI to create test designs for the same requirements and compared its design with our own. During this comparison, we identified flaws in the AI-generated design, which we corrected accordingly. Afterwards, we used the AI to generate all the unit tests and then manually reviewed them for correctness. We evaluated the generated tests based on our interpretation of the specifications and requirements.

- The AI generated a passing test for the requirement concerning the length of international phone numbers. We intervened because the requirement specified the correct length while the code did not correspond to the actual international phone number format.
- We identified a missing test case for email validation: checking that an email address containing two @ symbols is rejected.
- We refined our Wallet EP partitions to cover both invalid boundary conditions: values that are too low and values that are too high. The AI had only suggested testing a negative value.

4. Judgment

Where did *you* have to decide something the tools/AI couldn't decide for you? (e.g. which equivalence classes matter, whether coverage was "enough", whether a mutant was equivalent.)

We had to apply our own judgement in several areas where the AI fell short.

- Challenging AI "Green-washing": When our test for the international phone numbers failed, AI simply rewrote the test to force the test to pass. If we had not paid attention to the output of the AI chat we may have missed AI masking an underlying bug.
- Defining Boundary Sufficiency: While code coverage tools can show if a line is executed, they cannot judge if the test cases are logically sufficient. We had to decide which actually mattered. For example, proactively adding an upper bound validation limit for the Wallet, and identifying negative test cases (like multiple @ symbols in email validation) that the AI completely overlooked.

5. What we'd test next

We only covered part of the requirements, so we would have continued writing test cases for the more of the scenarios. In particular, we would add a test to verify that a new seeker cannot book a second concurrent active booking. This would ensure that we catch the off-by-one comparison bug (> vs. >=) that was identified in Activity 1.2.